

One Namespace, or One **Vote** Each

Nine patterns measured on six topologies in nine runnable shapes. Every card states the mechanism, what it buys, what it costs, and the check ID that proves it. One whole card is things this lab got wrong and corrected.

THE QUESTION EVERY CARD SERVES

Two regions, one platform. **Do you want one global stream name, or do you want each region to keep working when the other goes dark?** No shape in this lab gives you both.

THE RULE THAT ORDERS THE DECK

A **gateway** fuses two regions into one JetStream system. A **leaf link** gives each one its own. An **account** is the only real wall. A **domain** is a namespace, not a wall. Every card is a consequence of those four sentences.

CARD FAMILIES

■ Links — gateway, leaf node

■ Quorum — arbiters, and what a write is

■ Traps — both links at once

■ Boundaries — account, domain

■ Copying — mirrors, export / import

■ Retractions — what we got wrong

01

LINKS

A gateway buys one name and sells your survival

Two clusters of three, joined by a gateway, are **one JetStream system** with one meta group of six. A stream name is unique across both regions. You cannot opt out of that, and the price is a shared fate.

DECISION

Do both regions share one JetStream namespace? With a gateway, **yes, always** — and a three-three split leaves nobody with a majority.

MECHANISM



- › **A stream name is unique across the whole supercluster, not per cluster.** ZA creates `ODOMETER`; AU asking for the same name in the same account is refused with `10058` — `A3`, `A4`, `A5`.
- › **A per-cluster `jetstream.domain` does not rescue it.** It is illegal over a gateway: the supercluster stays one meta group and one of the two regions stops electing a leader altogether — `B1`, `B2`, `B7`.
- › Measured 2026-09-23: freeze AU and every create from ZA is refused with `10008 JetStream system temporarily unavailable` — `A11`. A read of AU's stream fails too — `A21`.

WHAT IT BUYS

- › **One name, one copy, no duplicates possible.** The shape cannot produce a split-brain of two same-named streams.
- › **A mirror needs no `external.api`.** One namespace, so a cross-region mirror just works and keeps up — `A15` — `A17`.
- › **Placement is predictable.** Everything lands where the account first asked, unless you name a cluster — `A9`, `A18`, `A19`.

WHAT IT COSTS

- › **Shared fate.** One meta group across the WAN, so a cut is a change freeze for everybody.
- › **Three plus three is the worst possible split.** Neither side can elect. Both regions freeze at once, not one.
- › **The dark region's data is unreachable,** not just unwritable — `A21`.
- › **No domain escape hatch.** The one thing that looks like a fix half-blinds a region instead.

VERDICT

Pick a gateway when **one global name** matters more than regional survival — and then add votes outside both regions (card 04), because the gateway on its own has no answer for a WAN cut.

02

LINKS

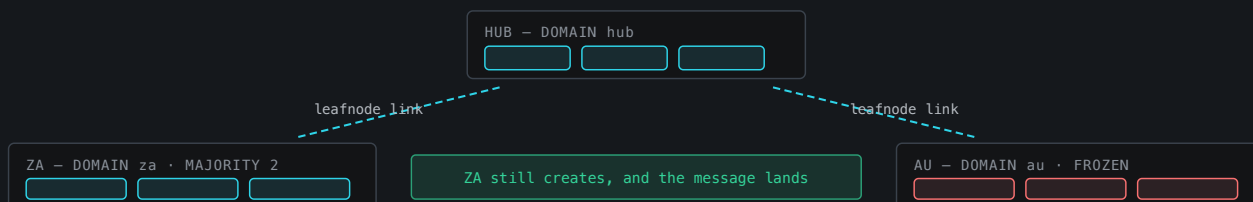
A leaf link buys a vote each and sells you a double copy

A hub with a leaf link to each region makes **three** JetStream systems, not one. Each region elects its own leader and owns its own names. Nothing crosses between them unless a person writes it down.

DECISION

Should each region be its own JetStream system? With a leaf link, **yes** — and the same stream name may then exist in all three at once, holding different data.

MECHANISM



- › Measured 2026-09-23: freeze AU and ZA still keeps a meta leader (D24), still creates a new three-replica stream (D25), and the message **really lands** — D26 counts it, it does not just publish it.
- › **Killing the hub outright changes nothing.** A region keeps its own JetStream and keeps storing — D14 , D23 .
- › **A bare stream name resolves in the local domain only.** The hub asking for ZA's ODOMETER by name simply fails, and may then make a third one of its own — D21 , D22 .

WHAT IT BUYS

- › **A vote each.** One region dark is one region dark, not a platform freeze.
- › **Survives the hub disappearing entirely.** The hub is a joiner, not a dependency.
- › **The same name in three places**, each owning its own data.

WHAT IT COSTS

- › **The double capture.** One publish on a shared subject is genuinely stored **twice**, once per region, and nothing warns you — D5 , D18 .
- › **Nothing copies itself.** Every cross-region copy is a mirror somebody has to write (card 06).
- › **A name is not global any more.** Anything that assumed one name, one stream must change.

VERDICT

Pick a leaf link when **regional survival** beats one global name. Then decide, in writing, which subjects may be captured in both regions — the shape will not decide it for you.

03

BOUNDARIES

The account is the wall. The domain is only a namespace.

Three things look like isolation: the **account**, the **cluster** and the **JetStream domain**. Only one of them is a wall. The other two decide *where* a thing lands and *what name* it answers to.

DECISION

Which boundary lets a region own its own stream? **The account**. Give each region its own account and the same name is two separate streams, over a gateway or a leaf link alike.

MECHANISM

- › **An account is a wall on purpose.** Two accounts cannot see each other's subjects, streams or consumers — not by name, not by accident — **A6**, **A7**, **A8**, **H3**, **H9**.
- › **A cluster is placement, not ownership.** A stream lands where the account's first request landed, unless you name a cluster with **--cluster**. A KV bucket follows the same rule, because a bucket *is* a stream — **A9**, **A18**, **A19**.
- › **A domain is a namespace with its own management group.** A bare stream name resolves only inside the asking domain. That is why the shared **LB** account under a leaf link ends up holding two streams of one name — neither side can refuse the other's overlap.
- › **And a domain is not a wall.** Name the other domain's API and data crosses it, which is exactly what the mirrors in **D8** and **H11** do. An account has no such door.
- › **Where a domain was measured:** four shapes only — **T2 / B**, **T5 / D**, **T5 / H**, **T6 / G**. T1, T3 and T4 carry no **jetstream.domain** at all, so nothing here claims a domain result on those three.

WHAT AN ACCOUNT BUYS

- › **Real ownership.** Each region's streams are invisible to the other, so a name collision is impossible rather than merely unlikely.
- › **It works under every link type in this lab** — gateway (**A6 – A8**), arbiter (**C14**, **F14**), leaf (**H3**).
- › **Sharing becomes deliberate**, through export / import only (card 07).

WHAT IT COSTS

- › **Nothing is shared by default.** Every cross-region read is a hole you design and maintain.
- › **Two accounts is two sets of credentials, limits and monitoring**, for ever.
- › **A domain over a gateway is a trap**, not a milder version of this — it changes nothing useful and blinds one region (card 01).

VERDICT

Use an **account per region** whenever a region must own its data. Use a **domain** only with a leaf link, and treat it as a naming decision. Never reach for a domain to get isolation.

04

QUORUM

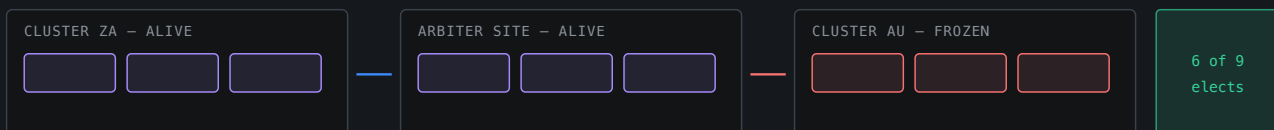
An arbiter site buys the vote back

The gateway's problem is arithmetic: three and three has no majority. Put votes in a **third place** and the arithmetic changes. One arbiter node makes seven, majority four. Three arbiter nodes make nine, majority five.

DECISION

How many arbiter nodes? **Three, not one.** With one, the arbiter itself is a single machine standing between you and a frozen platform.

MECHANISM



- › **Both directions are measured, not one.** Lose AU and ZA creates a three-replica stream (**F3** , **F4**). Lose ZA and AU does (**F22** – **F24**). The second direction was assumed for a long time; it is now a check.
- › **Five of nine is exactly the majority and still elects** — **F5** . Four of nine does not: no leader, and every create fails — **F7** , **F8** .
- › **A publish into a stream that already exists keeps working at four of nine**, and really stores — **F9** , **F21** . Both recoveries return the group to nine — **F10** , **F25** .
- › **The seven-server version works too** — **C4** , **C6** . It is the blast radius, not the behaviour, that differs.

WHAT IT BUYS

- › **The gateway's one name, without the shared freeze.** This is the only shape in the lab that keeps both.
- › **Symmetric survival.** Either region alone, measured both ways.
- › **Clean recovery.** The meta group comes back to full size on its own.

WHAT IT COSTS

- › **A third site.** Real money, real latency, real operations — for servers that hold no business data.
- › **The arbiter is not fenced.** Unplaced streams never chose it, but a stream *asked* to go there goes there, and a client dialling its port directly creates there — **C7** – **C11** , **F17** .
- › **Losing two of three regions still freezes you.** Four of nine is not a majority.

VERDICT

If you must have a gateway, **run a three-node arbiter** in a third site and fence it yourself with placement rules. A one-node arbiter is a demo, not a design.

05

QUORUM

“Can we still write?” is three questions

Most availability arguments about NATS go wrong here. **Creating** a stream needs the meta group. **Publishing** into a stream that already exists does not. **Reading the dark region's** stream is a third question again. They do not have the same answer.

DECISION

Which one does your service actually do during an outage? **Answer that first.** A platform that only publishes survives shapes that a platform which creates streams cannot.

MEASURED, CELL BY CELL — A BLANK CELL IS NOT A CLAIM

SHAPE	CREATE A STREAM	PUBLISH INTO AN EXISTING LOCAL STREAM	READ THE DARK REGION'S STREAM
T1 no link	yes, ZA dark T1h	not measured	impossible — no link
T2 gateway	no, 10008 A11	yes A12	no A21
T3 1-node arbiter	both ways C4 C6	not measured	not measured
T4 3-node arbiter	both ways F4 F23	yes F21 F24	not measured
T5 hub and leaf	yes, AU dark D25	yes, AU dark D26	not measured
T5 / H + account per region	hub loss only H13	hub loss only H16	not measured
T6 gateway AND leaf	no, 10008 G11	yes G12	not measured

> **Losing quorum is a change freeze, not a write freeze.** The client sees 10008 on any *change*, but only once the old leader has aged out — before that it simply hangs and times out.

> **One condition on that.** A publish keeps working while *that stream's own replicas* keep their own majority. Every cut here darkens a whole region, so the survivor's streams keep all three replicas in one live cluster. **Losing a single stream's own replicas is not measured here.**

> **A core nats pub proves nothing about storage.** It asks for no JetStream ack, so it exits zero even when the message was never stored. Any claim that a write landed must *count* it — A12 , F21 , D23 , G12 , H16 all do.

WHAT THE SPLIT BUYS YOU

> **An honest availability target.** Three numbers, not one.

> **Cheaper shapes become viable.** A publish-only service survives a plain gateway.

> **It names the real fix.** If you must create streams during an outage, you need votes outside both regions, or a domain each.

WHAT IT COSTS

> **You must know your own write path.** Auto-created streams and KV buckets are creates, and they will freeze.

> **A stale monitor hides the freeze** for about a minute (card 09).

> **Reading the dark side is unsolved in every shape here.** Only a mirror answers it (card 06).

06

COPYING

A mirror is the second copy — and it can copy the wrong stream

Nothing crosses a leaf link on its own. A **mirror** is how a region gets a readable copy of the other region's data. Over a leaf link it needs `mirror.external.api`. Leave that out and you get the worst outcome in the whole report.

DECISION

How does region B read region A's data? **A mirror, always named explicitly.** A mirror with no `external.api` does not fail — it reports healthy and quietly copies nothing, or the wrong thing.

MECHANISM

- › **Over a gateway it just works.** A supercluster is one namespace, so the mirror needs no `external.api` at all, and it keeps up — [A15](#) – [A17](#).
- › **Over a leaf link it must name the other domain's API** — `mirror.external.api = $JS.au.API`. With it, the messages copy: [D8](#) and [H11](#) both count **2 of 2**.
- › **Without it there is no error and no warning.** The stream reports healthy and sits at zero — [D7](#).
- › **Worse: if a stream of the same name exists locally, the mirror copies that one instead** and still looks correct. This run proved it by giving the two streams different message counts, so the number alone says which was copied — [D9](#), [D9a](#).
- › **The same trap catches export / import.** A mirror name resolves inside the *asking* account, so an imported subject does not make the other side's stream mirrorable by name — [E17](#), [E18](#).

WHAT IT BUYS

- › **A readable local copy** of the other region, which is the only answer in this lab to “read the dark region's stream”.
- › **It works in both link shapes**, with one config line of difference.
- › **It is explicit.** A mirror is a decision somebody made, and it is in the config where a reviewer can see it.

WHAT IT COSTS

- › **Silence on failure.** Health is not evidence. Only the message count is.
- › **Hand-written, per stream.** There is no “mirror everything”.
- › **Lag and bandwidth are not measured here.** Both shapes prove a mirror copies; neither says how fast or at what cost on the wire.

VERDICT

Always write `external.api` explicitly, even where it is optional. Then **alert on the mirror's message count, never on its health** — a wrong mirror is a healthy mirror.

07

COPYING

Export / import is the only sharing that is safe by design

Two accounts cannot see each other. **Export / import** is the one deliberate hole: the owner opens it, in one direction, for named subjects only — and the receiver renames whatever comes through.

DECISION

How do two regions share a subject on purpose? **An export from the owner and an import with a prefix.** The copy can then never be mistaken for the importer's own data.

MECHANISM

- › **One hole, one direction.** The exporting account names the subjects. Nothing else crosses, and the reverse direction is not implied — [E1](#) – [E5](#).
- › **The importer renames on arrival.** An imported subject arrives under the importer's own prefix, so local data and borrowed data are never the same name.
- › **It is a subject hole, not a stream hole.** The importer still cannot read, name or delete a stream the exporter owns, and may reuse that stream name for something entirely different — [E9](#).
- › **Which is exactly why a mirror by name still fails across it** — [E17](#), [E18](#). Subjects moved; names did not.

WHAT IT BUYS

- › **Sharing you can review.** The hole is in the config, named, and one way.
- › **No accidental collision.** The rename makes a borrowed subject visibly borrowed.
- › **The deliberate opposite of the double capture.** Card 02's trap is silent; this is loud.

WHAT IT COSTS

- › **Subjects only.** If you need the other region's *stream*, you need a mirror (card 06), and a mirror needs the domain named too.
- › **Per-subject work.** Every share is another pair of config entries to maintain and to audit.
- › **Two config files must agree.** An export with no matching import is silent, like every other silence in this lab.

VERDICT

Use export / import for **subjects a region genuinely needs from another**, and keep the list short. If the list grows long, the account split is in the wrong place.

08

TRAPS

Both links at once is not both shapes at once

The obvious idea: wire a gateway **and** a hub leaf link on the same servers, keep the gateway's single namespace, and get the leaf link's independent vote. It does not work. The **gateway wins, completely**.

DECISION

Can you combine the two link types to get both benefits? **No**. You get two JetStream systems, not three, and you pay for both links.

MECHANISM



- › **The duplicate stream name is refused again** — [G4](#). The leaf link's best property for the two regions is gone.
- › **The double capture collapses to a single stored copy** — [G6](#). So does its worst property, which is small comfort.
- › **A dark region still freezes every create** with [10008](#), while the hub sits beside it perfectly healthy and unable to lend a vote — [G11](#).
- › **Deleting the hub entirely changes nothing** for the regions — [G13](#). A per-site domain is still illegal here, exactly as with no hub at all.

WHAT IT BUYS

- › **The hub does keep a working JetStream system of its own.** It holds its own leader while a region is dark, and a client on the hub still creates a stream.
- › **Killing the hub leaves the regions untouched.** The hub is never a dependency.
- › **It settles the question once.** Nobody on the team has to wonder again.

WHAT IT COSTS

- › **Two links, two sets of failure modes, one benefit.** The first shape in the lab that costs real money and buys the two regions no quorum they did not already have.
- › **It reads as safer than it is.** The topology diagram looks redundant; the vote arithmetic is unchanged.
- › **More servers to run, patch and monitor**, for a hub the regions do not need.

VERDICT

Pick one link type. If you want regional votes, use a leaf link and drop the gateway. If you want one namespace, use a gateway and add an arbiter (card 04).

09

RETRACTIONS

What this lab got wrong, and corrected

A lab whose numbers only ever improve is not measuring. These are the results this rig published and later had to withdraw. Each one is kept beside the thing it damaged, because the **failure mode** is the transferable part.

DECISION

How much of a multi-region result should you trust on one reading? **Less than you think.** Four of the five entries below looked like healthy green results at the time.

RETRACTED — AND WHAT REPLACED IT

- › **A freeze that did not freeze.** An early harness recorded the wrong process id, so `kill -STOP` hit nothing and the “frozen” region stayed fully alive. The lab reported healthy behaviour and it looked like a finding. **Now:** every freeze proves itself first by checking the monitor port has really gone dark — [A10](#) , [A11](#) .
- › **A core publish read as a stored write.** Three checks called `nats pub` and read only its exit code. A core publish asks for no JetStream ack, so it exits zero even when nothing was stored. **Now:** every write claim counts the message — [A12](#) , [F21](#) , [D23](#) , [G12](#) , [H16](#) . Corrected 2026-09-23.
- › **The availability matrix cited healthy-state checks for an outage.** Two cells claimed a dark-region read had been measured, citing checks that ran with every server alive. **Now:** those cells read *not measured*. No T5 shape ever read a dark region — corrected 2026-09-23.
- › **“T4 was never built.”** Three documents said so for two days after `lab/06-arbiter3.sh` was measuring it. **Now:** card 04 is that rig, 23 checks, both directions of region loss.
- › **“A leaf link on a gateway buys nothing.”** Too wide. The hub does keep a working JetStream system of its own. **Now:** it buys the *two regions* no quorum — card 08.

THE TWO READINGS THAT LIE, AT BOTH EDGES OF A FAILOVER

- › **The old leader lingers.** After the majority of a supercluster was frozen, `/jsz` kept naming a leader for roughly a minute. Read it too early and a dead supercluster looks perfectly healthy — [A10a](#) , [A11a](#) .
- › **The new leader is announced before it is ready.** After an election the endpoint names the new leader some seconds before it will accept a create — [F6a](#) , [F7a](#) . The rig retries and records how long it waited, instead of sleeping a magic number.
- › **Which region goes blind is a coin toss.** A domain per cluster over a gateway blinds one of the two regions — and *which* one is not fixed. Two runs minutes apart, nothing changed, and the blind side swapped — [B1](#) , [B1a](#) , [B2](#) .

WHAT THE CORRECTIONS BOUGHT

- › **Cell-level honesty.** The report now says *not measured* in a cell rather than letting prose imply a result.
- › **A rig that fails loudly.** A recorded `FAIL` now fails the whole run, which it did not before.
- › **Reusable failure modes.** Stale monitors, silent mirrors and unacked publishes are the transferable lessons, not the counts.

WHAT THEY COST

- › **Some claims got smaller.** Several confident sentences are now scoped, and a few cells are blank.
- › **Re-running is the only proof.** Any figure not produced by `./run-all.sh` is hearsay, including the good 2026-09-11 hand-run results.
- › **Green is not evidence.** Four of these five were green when published.

VERDICT

Trust a multi-region result only when the rig **proved its own outage** and **counted its own write**. Everything else is a reading taken at the wrong moment.



GUIDE

Which card answers your question

The deck is ordered by mechanism. Most people arrive with a requirement instead. This page maps one to the other.

START HERE

IF YOU ARE ASKING

READ

THE SHORT ANSWER

"Can both regions keep working when the WAN cuts?"	05 , then 04 or 02	Split it into three questions first. Then buy votes or buy domains.
"Do we want one global stream name?"	01 vs 02	Gateway gives it. Leaf link takes it away and gives survival back.
"How do we stop regions colliding?"	03	An account per region. Not a domain.
"We have a gateway and cannot lose it."	04	Three arbiter nodes in a third site, fenced by placement rules.
"How does one region read the other's data?"	06 , then 07	A mirror with the domain named. Subjects only? Export / import.
"Can we run a gateway and a leaf link together?"	08	No. The gateway decides, completely.
"How much of this should we trust?"	09	Only what the rig re-runs. Green is not evidence.

STILL OPEN — DO NOT READ THESE AS ANSWERED

- › **The cost of a cross-region read in milliseconds.** Stream, consumer and KV placement are all measured. Latency is not.
- › **A real WAN partition.** Every region loss here is **kill -STOP** — a clean, total stop. Both sides alive and unable to reach each other is a different rig, and it is the one that shows reconnect and split-brain behaviour.
- › **Mirror lag and bandwidth.** Both shapes prove a mirror copies. Neither says how fast.
- › **The hub as a real store under load**, and what losing it costs then.
- › **Leaf reconnect with many regions.** Two were tested. Nothing tested five.

VERDICT

Two regions, one platform: **an account per region over a leaf link** if survival wins, **a gateway plus a three-node arbiter** if one global name wins. Nothing in this lab gives both.



METHOD

Where every number came from

A figure with no date and no machine becomes a constant, and a constant goes stale quietly. Every number in this deck is listed here with the day it was taken and the thing it ran on.

THE MACHINE

- > **nats-server v2.14.6** , bare processes on **127.0.0.1** . No Docker, no **nsc** trust chain, no **nats** contexts. Client **nats** CLI **0.4.0** . Host: Darwin 25.4.0 arm64.
- > **Four accounts, held still in every topology** — **LB** (shared), **LB_ZA** , **LB_AU** , **\$SYS** . Plain user / password pairs, lab-only. The account is never the variable; **the topology is**.
- > **A region is cut with `kill -STOP`** on that region's processes, and restored with **`kill -CONT`** . Every freeze proves itself dark before the rig asks anything.
- > **Every answer is read from `curl "localhost:8231/jsz?meta=1"`** , never from the server logs.

THE NUMBERS			
CARD	FIGURE	TAKEN	ON WHAT
all	175 checks passed, 0 failed, 51 observations	2026-09-23	lab/run-all.sh , all nine shapes from nothing
01	3 of 6 is no majority; every create 10008	2026-09-23	lab/01-gateway.sh , 26 checks
01	A domain per cluster blinds one region, side not fixed	2026-09-23	lab/02-domain-over-gateway.sh , 15 checks, two runs
02	AU dark: ZA creates, and 1 message really lands	2026-09-23	lab/04-hub-and-leaf.sh , 26 checks
03	Mirror copies 2 of 2 across a domain, with external.api	2026-09-23	lab/04-hub-and-leaf.sh D8 , lab/08-hub-leaf-per-region.sh H11
04	7 servers, majority 4 — both regions survive alone	2026-09-23	lab/03-arbiter.sh , 18 checks
04	9 servers, majority 5; 5 of 9 elects, 4 of 9 does not	2026-09-23	lab/06-arbiter3.sh , 23 checks, both directions
05	The three-question matrix, cell by cell	2026-09-23	all nine shapes; a blank cell was not run
07	Export / import moves subjects, never stream names	2026-09-23	lab/05-export-import.sh , 17 checks
08	Two systems, not three; hub loss changes nothing	2026-09-23	lab/07-gateway-and-hub.sh , 23 checks
09	The stale leader lingers about a minute	2026-09-23	lab/01-gateway.sh A10a , A11a

HOW TO READ THEM

- > **Read the shape, not the value.** Nine servers on one laptop is not your production estate. “Five of nine elects, four does not” transfers. Timings would not, which is one reason this deck carries none.
- > **A check has an expected answer and passes only on an exact match.** An observation with no expected answer is recorded as a *note* and can never pass or fail. The deck quotes checks.
- > **A check ID is permanent.** IDs are never reused and never renumbered, because they are cited outside this repo.
- > **Earlier results exist and are not quoted here.** The 2026-09-11 findings were real, but they were produced by hand in a terminal and are not re-runnable. Card 09 says why that matters.
- > **Reproduce all of it:** **`cd demos/03-multi-cluster-and-accounts/lab`** then **`./run-all.sh`** . It builds all nine shapes from nothing, tears them down, and rewrites **REPORT.md** and **REPORT.html** .